

Demystifying NVSHMEM: A System-Level Analysis on Symmetric Memory and Device-Initiated Operations in GPU Communication

Yijun Ma^{1*}, Siyuan Shen^{1*}, Tiancheng Chen¹,
Akhil Langer², Jiri Kraus², Benjamin Glick², Craig Belusar², Jeff Hammond², Torsten Hoeffler¹
¹ETH Zürich, Switzerland

{siyuan.shen, tiancheng.chen, torsten.hoeffler}@inf.ethz.ch, yijunma@student.ethz.ch

²NVIDIA Corporation, {alanger, jkraus, benjaming, cbelusar, jeffpapers}@nvidia.com

* These authors contributed equally to this work

Abstract—NVSHMEM is NVIDIA’s OpenSHMEM-based PGAS communication library for GPU clusters, enabling GPU-initiated, one-sided communication through symmetric memory. Despite its growing adoption, a system-level understanding of its design and behavior remains scattered across documentation, source code, and application experience. This paper presents a concise study of NVSHMEM’s programming model, implementation, and performance characteristics, focusing on symmetric memory, one-sided operations, and device-side collectives. We also examine DeepEP as a case study of NVSHMEM in performance-critical sparse deep learning workloads. Our analysis shows that NVSHMEM pioneered a device-side symmetric-memory programming model that enables fine-grained GPU-driven communication and is important for approaching the hardware performance limit. Overall, this work defines NVSHMEM’s role as a systems building block, highlights its design tradeoffs, and identifies opportunities for improving GPU communication runtimes.

Index Terms—NVSHMEM, PGAS, GPU communication, one-sided communication, collective communication

I. INTRODUCTION

The rise of distributed deep learning and GPU-accelerated scientific computing has made GPU communication a central challenge. Distributed deep learning increasingly relies on different types of parallelism, such as data, tensor, and pipeline parallelism, to scale training and inference across many GPUs and often multiple nodes [1]. HPC applications likewise rely on distributed decompositions to scale simulations beyond a single device [2]–[4]. In both settings, performance depends on how efficiently data can be exchanged over interconnects such as PCIe, NVLink, and InfiniBand (IB). Communication libraries must therefore provide high bandwidth, low latency, and abstractions that integrate naturally with GPU execution. The NVIDIA Collective Communication Library (NCCL) [5] emerged as the most widely used solution, providing optimized collective operations and serving as the recommended backend for most CUDA-based distributed machine learning frameworks such as PyTorch [6], Megatron-LM [7], and vLLM [8].

Originally, NCCL was host-driven: the CPU enqueues collective operations, and the library selects algorithms and schedules. While effective for bulk-synchronous collectives,

this model is less suitable for fine-grain point-to-point communication. Such patterns arise when communication is data-dependent, restricted to subsets of threads, or when CPU coordination leads to unacceptable latencies. Examples include stencil halo exchange, irregular graph communication, sparse and expert-parallel workloads, and custom kernels that tightly couple computation and data movement.

NVSHMEM [9], [10] addressed this gap through a complementary programming model. It exposes a partitioned global address space (PGAS) directly to GPU code, enabling CUDA kernels to access symmetric memory on remote processing elements via one-sided put/get operations, atomics, and explicit synchronization primitives. Rather than replacing NCCL, NVSHMEM enabled communication patterns that are difficult to express with host-launched collectives, including irregular exchanges and fine-grained overlap.

As NVSHMEM proved successful across several workloads, NCCL incorporated related ideas through its new device API, adding device-initiated operations and symmetric memory support [11]. These features build on concepts pioneered by NVSHMEM and reflect lessons from its use in AI workloads, but the two designs target different abstractions. NVSHMEM provides a flat remote-memory view while NCCL makes the distinction between scale-up and scale-out domains explicit. NCCL’s device API also addresses a key limitation of NVSHMEM: the lack of hierarchical composition through communicators. Although NVSHMEM supports teams, its OpenSHMEM roots still expose a single symmetric heap shared across all GPUs in an NVSHMEM instance.

Despite NVSHMEM’s growing importance, many of its key behaviors are not apparent from the API alone. How does NVSHMEM realize symmetric memory on modern GPUs? Which communication paths avoid CPU involvement, and when are they used? What mechanisms underlie its collectives? This paper answers these questions through a source-level analysis of NVSHMEM, tracing its design from the programming model to runtime internals and transport mechanisms. While comparisons with NCCL appear throughout the paper, they are intended to clarify design differences rather

than to provide an official or definitive ranking of the libraries.

Our analysis primarily targets NVSHMEM version 3.3.9. Although future releases may change specific features or implementation details, the core architectural components, such as symmetric memory and communication paths, are fundamental to NVSHMEM’s design and are likely to remain largely unchanged. We therefore expect the main insights of this paper to remain relevant for future versions as well.

II. BACKGROUND

A. Essential Concepts

To better understand NVSHMEM, it is first necessary to introduce several core concepts that shape its design. NVSHMEM builds on three main ideas: the limitations of host-initiated communication, the motivation for device-initiated communication, and the PGAS/OpenSHMEM programming model based on symmetric memory and one-sided operations.

NVIDIA Collective Communication Library (NCCL) has become the standard communication library for multi-GPU systems, but its conventional interface remains largely **host-initiated** [5]. In particular, communicator setup and the launch of collective operations are performed from the CPU, which then schedules NCCL kernels onto CUDA streams. While this model is highly optimized for regular collective communication, it still places the host in the communication control path and can introduce coordination overhead between computation and communication.

This limitation motivates **device-initiated communication**, where communication and synchronization are launched directly from GPU code rather than through a host-managed control path. This reduces CPU involvement and enables finer-grained overlap between communication and computation. These benefits are especially important for latency-sensitive and strongly scaled GPU workloads, and motivate interfaces such as NVSHMEM that support GPU-side communication primitives and GPU-initiated network operations that bypass CPU proxy threads completely [9], [12], [13].

Peer-to-peer (P2P) memory access refers to the ability of one GPU to directly access another GPU’s memory. This use of P2P should be distinguished from point-to-point communication and from peer-to-peer network architectures such as Gnutella. This enables low-overhead remote memory access within a **scale-up** domain, usually a node-level NVLink/NVSwitch domain, but potentially a multi-node NVLink (MNNVL) domain. **GPUDirect RDMA (GDRDMA)** [14] extends direct GPU memory access to **scale-out** domains by allowing an RDMA-capable NIC to read from and write to GPU memory. **GPUDirect Async Kernel-Initiated (GDA-KI)** [11], [15] further allows GPU kernels to initiate and control network operations without CPU intervention. **InfiniBand GPUDirect Async (IBGDA)** [12] is the InfiniBand-specific realization of GDA-KI.

The **Partitioned Global Address Space (PGAS)** model exposes a logically shared address space partitioned across processing elements (PEs) [16]. This model underlies OpenSHMEM [17] and is adapted by NVSHMEM for GPU-based

systems. It is also closely related to one-sided communication in MPI-3 RMA [18], where a process can access memory exposed by another process without an explicit matching receive. The main difference is in the interface and runtime model. OpenSHMEM and NVSHMEM provide a lighter-weight call path by encoding operation properties, such as data type, transfer size, and operation kind, directly into the API name. MPI RMA, in contrast, exposes a more general interface where these properties are passed as arguments. This SHMEM-style interface can reduce dispatch and argument-processing overhead, which is useful for fine-grained communication.

In NVSHMEM, a PE is an OS process mapped to one GPU; since version 2.4.1, NVSHMEM also provides limited support for multiple PEs per GPU. Communication is built on **symmetric memory**, consisting of a **symmetric heap** on each PE and **symmetric objects** allocated with the same type, size, and layout across all PEs. This structure allows remote PEs to access those objects through one-sided operations such as `put` and `get`, where the initiator specifies the source, destination, and location without explicit receives from the target PE. This enables asynchronous progress and, in many cases, direct placement into the destination’s symmetric memory buffer.

B. GPU Communication Libraries

To understand NVSHMEM’s role in the broader high-performance computing (HPC) landscape, we compare it with existing GPU communication frameworks and libraries.

CUDA-aware MPI [19] extends the standard MPI programming model [20] by allowing MPI routines to operate directly on GPU-resident buffers, avoiding explicit application-level staging through host memory. This makes it an effective path for porting existing distributed-memory HPC applications to GPU clusters while preserving familiar rank-based, two-sided communication semantics [20]. However, CUDA-aware MPI remains fundamentally host initiated.

NCCL, discussed previously, is the main alternative to NVSHMEM for GPU communication. It is optimized for dense and regular collective communication [5]. This makes it the natural baseline for distributed deep learning and other bulk-synchronous workloads. However, its conventional interface was host-centric. Recent NCCL Device API work reduces this gap by introducing GPU-initiated communication, but the use of NCCL in the AI ecosystem remains collective-centric. NVSHMEM was designed around fine-grained one-sided operation and currently possesses functionality not found in NCCL’s GPU-Initiated Networking (GIN) interface [11].

UPC++ [21] and **GASNet-EX** [22] provide portable PGAS-style communication and share with NVSHMEM the idea of one-sided access to a partitioned address space. Their portability and C++ integration make them attractive for many host-driven HPC applications. The main distinction, however, lies in where communication is initiated. While UPC++ and GASNet-EX can support GPU memory through device-aware transports, the control path is still typically managed by host code. NVSHMEM, by contrast, exposes device-callable

TABLE I
BREAKDOWN OF NVSHMEM API GROUPS BY AVAILABILITY AND FUNCTIONALITY IN THE SOURCE CODE.

API group	Host/Device Availability	Description
Setup / exit	Mixed	Initialize and finalize the library, bootstrap jobs, and handle exceptional termination.
Memory management	Host-side	Allocate, free, and align symmetric objects in the symmetric heap, provide registration functionality for selected local buffers.
Team management	Mixed, mostly host-side	Define communication subgroups, query team-relative PE identities, translate PE numbers across teams, and create or destroy user-defined teams.
One-sided RMA	Both	Provide one-sided data movement through <code>put</code> , <code>get</code> , scalar <code>p/g</code> , etc.
Atomics	Both	Provide remote atomic read-modify-write operations such as <code>fetch</code> , <code>add</code> , <code>compare-and-swap</code> , and bitwise updates on symmetric data.
Memory ordering	Both	Provide ordering and completion primitives such as <code>fence</code> and <code>quiet</code> .
Synchronization	Mixed, mostly device-side	Provide value-based synchronization through <code>wait</code> and <code>test</code> operations.
Collectives	Both	Provide coordinated multi-PE operations such as <code>AllReduce</code> , and <code>Broadcast</code> .

operations that allow CUDA threads, warps, or thread blocks to initiate communication directly from the GPU.

rocSHMEM [23] and **Intel SHMEM** [24] bring OpenSHMEM-style, GPU-oriented one-sided communication to AMD ROCm and Intel oneAPI/SYCL systems, respectively. Together, they show that device-initiated PGAS communication is becoming a broader vendor trend rather than a feature unique to NVIDIA. However, each implementation is necessarily shaped by its own compiler stack, memory model, interconnects, and transport mechanisms. GICC [25] implements GPU-initiated communication for the Slingshot network using triggered operations.

III. NVSHMEM OVERVIEW

Before analyzing NVSHMEM’s internals, we first summarize the main concepts that shape how it is used in practice.

A. Teams

Teams are NVSHMEM’s mechanism for defining subsets of processing elements (PEs) that participate in communication. Unlike MPI or NCCL communicators, teams do not contain most of the communication-related runtime state. Instead, they are lightweight handles that identify PE groups and may encode topology information. The default team is `NVSHMEM_TEAM_WORLD`, which contains all PEs and is implicitly used whenever a team argument is omitted.

Teams carry both semantic and runtime constraints. A collective on a team involves exactly the PEs in that team, and team-relative operations must translate PE indices correctly. Arbitrary teams can also be constructed at runtime from existing PEs. However, a team cannot safely be used by multiple concurrent collective invocations from the same PE because the runtime associates a single set of internal resources, such as synchronization state and scratch storage, with each team.

B. API and Dual-Interface Design

This dual-interface design is one of NVSHMEM’s key strengths, but it also makes the library substantially more complex than communication libraries built around a single control path. This complexity is compounded by the fact that

the split is not uniform across the API: some components remain fundamentally host-managed, whereas the core communication path is exposed on both the CPU and the GPU. To clarify this structure, we group the APIs into functional subsets and summarize their purpose and availability in Table I.

In addition, the naming convention distinguishes the OpenSHMEM interface from NVSHMEM-specific extensions: `nvshmem_` generally denotes the standard API, whereas `nvshmemx_` is used for extensions such as stream-ordered operations and threadgroup-scoped variants. Internal helper functions often use the `nvshmemi_` prefix, where the `i` denotes internal routines that are not part of the public API.

Within the device-side APIs, NVSHMEM distinguishes operations by execution scope. The default APIs are thread-scoped, where a single CUDA thread issues the operation. Many `nvshmemx_` extensions also provide `_warp` and `_block` variants, where the calling warp or block collectively provides the resources needed to execute one operation.

We present an overview of the functionality of each API group as follows. For a complete description, we refer the reader to the official NVSHMEM documentation [9].

1) *Setup / exit*: This group brings NVSHMEM up and tears it down. It covers initialization, bootstrap through MPI or unique IDs, and finalization, and is therefore primarily responsible for creating the runtime state required by all later operations. In the source, this group is mostly host-side, with `nvshmem_global_exit` as the main exception that is also exposed on the device.

2) *Memory management*: The memory-management APIs allocate and manage symmetric objects in the symmetric heap. They include routines such as `nvshmem_malloc`, `nvshmem_align`, and `nvshmem_free`, together with buffer-registration functionality for selected local buffers. These routines remain host-driven because they manipulate global runtime state and collective memory layout.

3) *Team management*: Team-management APIs define communication subgroups and provide the rank-translation needed to operate within them. Query functions such as `nvshmem_team_my_pe` are available on both host and

device, while team creation and destruction remain host-managed.

4) *One-sided remote memory access (RMA)*: The RMA group is the core one-sided communication interface of NVSHMEM. It includes typed and untyped put/get operations, scalar p/g, strided iput/iget, and nonblocking variants. The API appears in several generalized naming forms: typed routines follow patterns such as `nvshmem_<TYPENAME>_{put,get}`, size-specialized routines follow forms such as `nvshmem_{put,get}<SIZE>`, and untyped byte-oriented routines use names such as `nvshmem_{putmem,getmem}`. Scalar p/g operations cover single-element access, while iput/iget provide shorthand for strided communication patterns. This group is fully dual-interface and is also extended with device-side warp- and block-scoped variants and host-side on-stream forms.

5) *Atomics*: Atomic APIs provide remote read-modify-write operations on symmetric data, including operations such as fetch and add. Their purpose is to support fine-grained remote coordination on individual memory locations. The API follows generalized naming forms such as `nvshmem_<TYPENAME>_atomic_<op>` for non-fetching operations and `nvshmem_<TYPENAME>_atomic_fetch_<op>` for fetching variants, along with special forms such as `compare_swap`, `swap`, and `fetch`.

6) *Memory ordering*: The memory-ordering interface is centered around a small number of key routines. `nvshmem_fence` enforces the ordering of previously issued communication operations to the same destination PE but does not, by itself, guarantee their completion. `nvshmem_quiet` is stronger: it waits until previously issued NVSHMEM communication operations from the calling context have completed and become visible at the destination. On the host side, NVSHMEM also provides `nvshmemx_quiet_on_stream`, which places this completion semantics into CUDA stream order.

7) *Synchronization*: The synchronization group lets a PE wait for or test remote progress based on the value of a symmetric object or signal variable. This includes the `wait` and `test` families as well as signal-based waiting operations. This group is less uniformly dual-interface: the ordinary `wait/test` routines are primarily device-side, while the host mainly accesses this functionality through on-stream variants.

8) *Collectives*: The NVSHMEM collective API group provides team-wide communication and synchronization operations, including Barrier, Sync, Broadcast, AlltoAll, FCollect, Reduce, and ReduceScatter. These APIs are available on both host and device. FCollect is the counterpart of MPI AllGather, while NVSHMEM Reduce is the counterpart of MPI AllReduce. In addition, NVSHMEM memory-management and team-creation functions are also collective and internally invoke `nvshmem_barrier_all`.

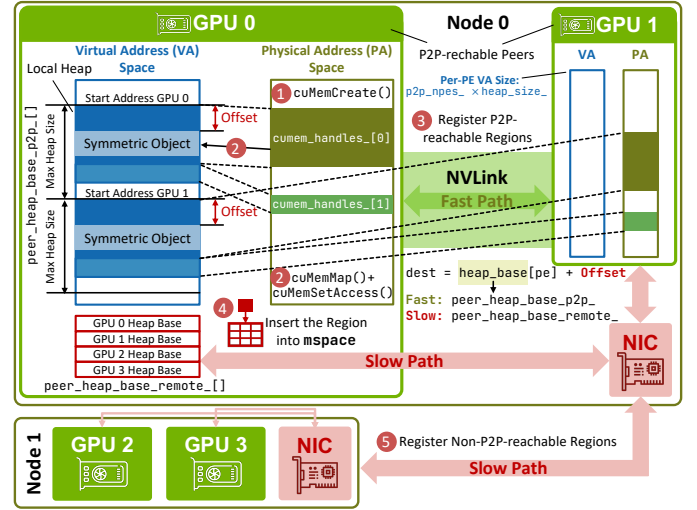


Fig. 1. Virtual-memory-based symmetric-heap setup in NVSHMEM for two nodes, each with two GPUs and one NIC. The numbered markers ①–⑤ show the main steps taken when new physical memory is allocated.

IV. MEMORY MANAGEMENT

Having outlined the overall design of NVSHMEM, we now examine its internals in more detail. We begin with how symmetric memory is realized and managed, as it is one of the core foundations enabling device-initiated communication.

NVSHMEM implements the symmetric heap on top of CUDA's low-level virtual memory management (VMM) API, and alternative backends using system memory or pinned device memory can be selected via environment options. With VMM enabled, NVSHMEM implements memory management scheme that cleanly separates **virtual address (VA) reservation** from **physical memory allocation**. The symmetric heap's VA range is reserved eagerly during initialization, while physical pages are committed on demand as the application allocates from the heap [26], [27]. Figure 1 illustrates the key concepts discussed in this section.

A. Initialization

During `nvshmem_init`, NVSHMEM creates a symmetric heap for each PE. It first chooses the allocation granularity and sets the per-PE heap size to the larger of `MAX_MEMORY_PER_GPU` and the internal runtime memory overhead, rounded up to the chosen granularity. Each PE then reserves a contiguous VA range of size `p2p_npes_ × heap_size_` using `cuMemAddressReserve()`, where `p2p_npes_` is the number of P2P-reachable GPUs. The first segment is used for the local heap, while the remaining equal-sized segments are reserved for peer mappings. NVSHMEM then creates a host-side `mspace` allocator without committing physical memory. After transport initialization, PEs exchange their local heap base addresses so that remote memory access can be established across nodes.

The reserved VA range covers only P2P-mappable peers. For non-P2P peers, NVSHMEM instead allgathers each PE's `heap_base_` into a separate table, `peer_heap_base_`

`remote_`, which stores the actual base address of every remote heap. The slow path uses this table together with transport metadata and remote memory handles.

B. Memory Allocation & Registration

On top of the reserved VA range, NVSHMEM manages the heap with a simple host-side allocator. It maintains three `std::map` structures: two for free chunks indexed by start and end addresses, and one for allocated chunks indexed by the start address. This design makes coalescing straightforward since freeing a block allows the allocator to check both neighboring directions and merge adjacent free chunks when possible. Allocation is performed using a first-fit policy: the free list is scanned in address order, the first sufficiently large block is chosen, and it is either split or consumed entirely. All sizes are aligned to `NVSHMEMI_MALLOC_ALIGNMENT`, and the address is returned to the user.

The allocator is deliberately simple and deterministic. It linearly scans the free list, and allocation is $O(n)$ in the number of free chunks. This overhead is acceptable as allocations are typically executed during initialization. In contrast to allocators such as `jemalloc`, it avoids per-thread arenas, size classes, or other complex optimizations.

NVSHMEM allocates physical GPU memory for the symmetric heap only on demand, when an allocation request cannot be satisfied from already-backed VA space. When a symmetric allocation request triggers the growth of the heap, `allocate_physical_memory_to_heap()` performs the following steps:

- ① Create a physical memory handle with `cuMemCreate()`.
- ② Map the corresponding heap subrange with `cuMemMap()` and install access permissions with `cuMemSetAccess()` for the local and eligible peer GPUs.
- ③ Register the region with P2P transport.
- ④ Insert the region into the `mSPACE` allocator.
- ⑤ Register the region with network transports for communication with non-P2P-reachable peers.

NVSHMEM then records the allocation metadata (i.e., offset, size, etc.), exchanges the required handles across PEs, and synchronizes with a barrier.

C. Mapping and Remote Address Computation

NVSHMEM reserves a VA range large enough to cover the symmetric heaps of all P2P-reachable GPUs and places each peer heap at a fixed offset within that range. This organization is central to the design, because it allows remote addresses to be derived from heap-relative offsets.

At a high level, it exports the local CUDA memory handle, exchanges these handles across PEs, and maps peer memory into the corresponding reserved virtual address segments. For non-P2P transports, the same region is additionally registered with the appropriate network interface.

Remote address computation follows the same symmetric-offset rule in both the fast and slow paths, the only difference is which per-PE base address is used and whether the resulting address can be dereferenced directly. In the P2P fast path,

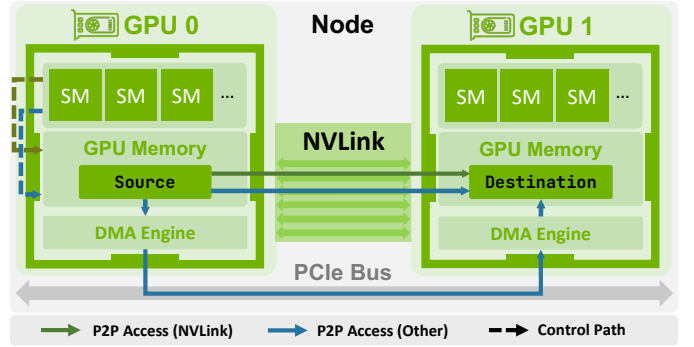


Fig. 2. Illustration of the device-side RMA fast path. Solid arrows denote data paths, while dashed arrows denote control paths, of the same color highlight the control path. Although the figure shows NVLink within a node, the same path applies beyond a single node when using MNNVL.

if `heap_base_` is the base of the local symmetric heap and `peer_heap_base_p2p_[remote_pe]` is the base at which the remote PE's heap is mapped in the local VA space, then for a local symmetric pointer `dest_local` the corresponding remote address is

$$\text{dest_remote} = \text{peer_heap_base_p2p_}[\text{remote_pe}] + (\text{dest_local} - \text{heap_base_}) \quad (1)$$

NVSHMEM, therefore, preserves the object's offset within the local heap and applies it to the remote heap base. For non-P2P-accessible GPUs, the same rule is applied using `peer_heap_base_remote_[remote_pe]`, and the computed address is passed to the transport layer together with the required memory handle, registration metadata, or NIC key. The same addressing scheme thus applies in both paths, but only the P2P case gives the GPU a directly mapped peer virtual address. NVSHMEM's device-side fast path uses this mapping for SM-issued loads and stores from within the kernel, which is distinct from host-initiated CUDA peer copies, which may use asynchronous copy engines depending on the GPU and interconnect. Whenever the heap grows, NVSHMEM also refreshes device-resident metadata so that GPU kernels observe the updated layout correctly.

V. ONE-SIDED COMMUNICATION

Having discussed how NVSHMEM manages memory, we now turn to another fundamental aspect of the library: one-sided communication. As noted in the previous section, NVSHMEM uses either a fast path or a slow path depending on whether the target peer is P2P-reachable. The following sections explain how data is moved between PEs in each case.

A. Fast Path: Direct GPU Memory Access

Fast-path eligibility. This decision is made by the host-side P2P transport logic. Figure 2 illustrates the two common cases that follow from this decision. For brevity, we focus on the most common case and omit less typical fallback paths, such as NVIDIA Management Library (NVML)-based handling. In this path, NVSHMEM verifies that the two PEs are on the same host, identifies the peer as a locally visible CUDA device, and uses `cudaDeviceCanAccessPeer` to

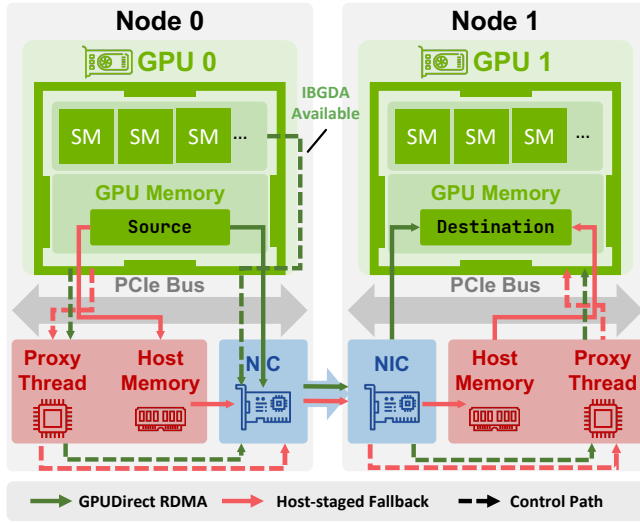


Fig. 3. Illustration of device-side RMA slow path. Solid arrows denote data movement, while dashed arrows of the same color highlight the control path.

determine whether direct peer access is supported. If P2P access is available, it queries whether native GPU atomics can be enabled. Only then does NVSHMEM populate `peer_heap_base_p2p_[pe]` for that PE.

Device-side RMA Once the peer heap is directly mapped, device-side RMA reduces to ordinary memory access on the computed remote address from Equation 1. For scalar operations, helpers such as `nvshmemi_p` and `nvshmemi_g` first check whether the target PE has a mapped heap base and then issue a direct store or load on the resulting remote pointer. Similarly, bulk interfaces such as `nvshmemi_put_threadgroup` perform threadgroup-wide `memcpy` operations to or from that mapped address.

Host-side RMA Host-side RMA uses the same mapped address space, but the execution model is different since the host orchestrates CUDA copy operations. Public APIs such as `nvshmem_put` and `nvshmemx_put_on_stream` delegate to `nvshmemi_prepare_and_post_rma`, which selects a mapped P2P path when the peer heap is locally accessible. In that case, NVSHMEM uses CUDA copy operations such as `cudaMemcpyAsync` between local and remote device pointers. For device-local symmetric buffers, it may use specialized helpers such as `nvshmemi_p2p_rma_optimized`, which select explicit copy kinds for stream-ordered and single-word operations.

B. Slow Path: IBGDA and Proxy Execution

The slow path is used when the target PE’s symmetric heap is not directly P2P-mapped into the local GPU address space. This occurs for inter-node communication and for intra-node peers that are not CUDA P2P-accessible. In this case, NVSHMEM still preserves the same offset-based addressing scheme described earlier, but the computed remote address can no longer be dereferenced directly by the local GPU. Instead, the operation is carried out through a network transport, as illustrated in Figure 3. Depending on system support, NVSH-

MEM either uses InfiniBand GPUDirect Async (IBGDA), if available, or a **host proxy** path, where a CPU thread executes the request on behalf of the GPU.

Network transport and connection management. Slow-path RMA relies on a pluggable remote-transport layer rather than on a single network interface model. NVSHMEM can select among InfiniBand-oriented transports such as IBRC and IBDEVX, as well as higher-level backends such as UCX and libfabric. The InfiniBand-specific transports maintain PE-to-PE connectivity through Reliable Connection (RC) queue pairs (QPs), while IBGDA additionally supports Dynamic Connection (DC) queue pairs.

However, this RC-QP description does not apply universally. For example, on HPE Slingshot systems, NVSHMEM uses the libfabric transport with the `cx` provider, where connectivity is established through libfabric endpoints. UCX similarly constructs its own PE-to-PE endpoints from exchanged worker addresses. Recent NVSHMEM versions also expose QP-specific APIs to users, allowing finer control over QP selection and usage when IB-specific paths are active [9].

Device-side RMA On the device side, scalar put/get operations use helpers such as `nvshmemi_transfer_rma_p` and `nvshmemi_transfer_rma_g`; bulk operations use `nvshmemi_transfer_rma` and `nvshmemi_transfer_rma_nbi`; and related operations such as signals and atomics are handled through specialized helpers such as `nvshmemi_transfer_put_signal` and `nvshmemi_transfer_amo_*`.

Each operation first checks whether IBGDA is available on the device. If so, the request is offloaded to routines such as `nvshmemi_ibgda_rma_*`, which construct and post RDMA work directly from GPU code to the NIC. Otherwise, the operation is encoded as a work request and written into a proxy buffer in host-pinned memory. A dedicated host proxy thread then consumes this descriptor and executes the corresponding operation via functions such as `nvshmemi_proxy_rma_p` and `nvshmemi_proxy_rma_g`. Thus, the slow path remains GPU-initiated at the API level, but completion is ultimately driven either by the NIC through IBGDA or by the host proxy through the descriptor queue.

Host-side RMA Host-side RMA APIs, such as `nvshmem_put`, delegate to the common helper `nvshmemi_prepare_and_post_rma`, which selects the appropriate transport. On the slow path, in short, the operation is treated as a remote network transfer.

In the off-stream case, NVSHMEM directly selects a remote transport with RMA capability, such as IBRC or libfabric, constructs the corresponding local and remote memory descriptors, and posts the operation through the transport’s host-side RMA entry point. Bulk transfers are handled through `nvshmemi_process_multisend_rma`, which posts the required sequence of RDMA operations. In the on-stream case, the request is instead encoded into the proxy command structure and launched on the specified CUDA stream through `nvshmemi_proxy_rma_launcher`, after which the host

TABLE II

SUMMARY OF COLLECTIVE ALGORITHMS IN NVSHMEM (v3.3.9). M IS THE MESSAGE SIZE, N IS THE NUMBER OF PES, AND $S = \lceil M/B_{\text{seg}} \rceil$ DENOTES THE NUMBER OF SCRATCH-LIMITED SEGMENTS IN SEGMENTED ALLREDUCE. ALGORITHMS WHOSE NAMES CONTAIN NVLS RELY ON NVSWITCH AND NVLINK SHARP TO PERFORM IN-NETWORK OPERATIONS.

Collective	Algorithm	LL / LL128	Communication Volume	Latency, # Synchronizations
Broadcast	Bruteforce put-to-all	No	$O(MN)$	$O(1)$
	k -ary flat tree	LL only	$O(MN)$	$O(\log_k N)$
	Topology-aware hierarchical tree	LL only	$O(MN)$	$O(\log_k N_{\text{remote}} + \log_k N_{\text{intra}})$
AlltoAll	P2P / general all-push	No	$O(MN^2)$	$O(1)$
FCollect (AllGather)	NVLS one-shot	LL only	$O(MN^2)$	$O(1)$
	Generic all-push	Both	$O(MN^2)$	$O(1)$
Reduce (AllReduce)	NVLS one-shot	No	$O(MN^2)$	$O(1)$
	NVLS two-shot	No	$O(MN)$	$O(1)$
	k -ary recursive exchange	No	$O(MN \times k \log_k N)$	$O(\log_k N)$
	Hierarchical fcollect	Inherited from FCollect	$O(MN^2)$	$O(1)$
	(Segmented) linear AllReduce	No	$O(MN^2)$	Direct LD/ST: $O(1)$; Segmented: $O(SN)$
ReduceScatter	NVLS one-shot	No	$O(MN^2)$	$O(1)$
	Generic all-push	No	$O(MN^2)$	$O(1)$

proxy thread drives the actual network operations..

The host-side slow path has several remote-transport limitations: remote strided RMA and host-side g operations are unsupported, and `put_signal` is available only through the on-stream proxy path. These limitations reflect that the current remote transport and proxy mechanisms are primarily optimized for contiguous transfers and device-side signaling.

VI. COLLECTIVE COMMUNICATION

Having established the mechanisms for symmetric memory and one-sided communication, we next examine NVSHMEM’s collective operations. These operations are built on top of the lower-level components discussed above.

NVSHMEM implements a variety of collective algorithms to target different message sizes and system configurations. Due to space constraints, we do not discuss each algorithm in detail. Instead, Table II summarizes the main designs and, based on our analysis, reports their communication volume and latency, measured by the number of synchronization steps. In the remainder of this section, we therefore focus on the core concepts that underlie NVSHMEM’s collective implementation rather than on each algorithm individually.

A. pSync Buffer

Each team in NVSHMEM owns a `pSync` buffer, short for **persistent synchronization buffer**, which is a symmetric memory region that is used to coordinate collective operations, and in some cases point-to-point operations, across PEs. NVSHMEM lays out `pSync` regions in a strided manner across teams so that synchronization states belonging to different teams do not fall on the same cache lines. Within each team’s `pSync` region, different collective operations are assigned fixed subregions using hardcoded offsets. In addition, some operations are double-buffered so that consecutive invocations can alternate between buffers and avoid requiring an additional barrier between uses.

B. Low Latency (LL) & LL128 Protocols

To reduce the cost of explicit synchronization after data transfer, which can become a major bottleneck for small messages, NVSHMEM implements Low Latency (LL) and LL128 protocols, following a similar design as NCCL [5]. The central idea of these protocols is to couple data movement with lightweight arrival notification.

In the LL protocol, each data unit is paired with synchronization flags: the sender packs two data elements and two flags into a single 16-byte atomic write, and the receiver polls the flags to determine when the data is ready. LL128 uses the same idea with a larger transfer unit, grouping 120 bytes of data with an 8-byte flag into 128 bytes to improve bandwidth utilization. However, LL128 is only safe over NVLink because it relies on 128-byte atomic stores, which are not generally guaranteed on interconnects such as PCIe [5].

The LL and LL128 protocols require additional `pSync` storage, since the reception buffer must accommodate both payload data and the flags. As not all algorithms support these protocols, Table II highlights the ones that do.

C. Algorithm Selection

In general, NVSHMEM selects collective algorithms at runtimes. Instead of using an explicit analytical model, NVSHMEM implements this choice as a rule-based decision tree for each collective. Capability checks, datatype and scope constraints, scratch-space availability, and fixed message-size thresholds determine which algorithm is allowed and preferred, with unsupported cases falling back to more general implementations.

D. Support for Multiple Cooperative Thread Arrays (CTAs)

The device-side collective APIs in NVSHMEM are thread-group collectives: they provide thread-, warp-, and block-scoped entry points, but a single collective invocation is executed by **one participating threadgroup** rather than by

multiple CTAs. The absence of public `_grid` variants is deliberate. A grid-scoped operation would require synchronization across CTAs inside a running kernel, which is more expensive than ending it and launching a stream-ordered communication kernel. This design is also consistent with NVSHMEM’s team model, where each team owns one set of internal collective resources such as `pSync` space, so a team cannot safely host multiple concurrent collective invocations without duplication.

NVSHMEM exposes built-in multi-CTA execution only through the host-side `_on_stream` wrappers, and only for `FCollect`, `AllReduce`, and `ReduceScatter`. On the first multi-CTA call, NVSHMEM creates duplicate teams in a per-team `team_dups[]` array, copies them to the device, and assigns each CTA a separate team and payload slice. Each duplicate team has its own `pSync` region, preserving the rule that one team cannot be used by concurrent collectives. This path is enabled only when NVLS resources are available; otherwise, NVSHMEM uses a single CTA or may fall back to NCCL when NCCL support is enabled. Thus, built-in multi-CTA execution is gated by NVLS availability. Users needing more parallelism typically issue independent block-scoped operations from multiple CTAs or launch a separate communication kernel through an on-stream API.

E. Barrier and Synchronization

NVSHMEM’s synchronization collectives are implemented separately from the data collectives in Table II. In the device code, both `nvshmem_sync` and `nvshmem_barrier` invoke a dissemination-style routine over the team’s `pSync` region. At each phase, a PE signals a set of PEs and then waits for the corresponding notifications from another set of partners. The radix k is chosen from the runtime parameter `barrier_tg_dissem_kval` and clamped by team size. For block-scoped collectives on fully P2P-connected teams, k may be raised to the team size. Under this structure, synchronization requires $\log_k N$ rounds for a team of N PEs.

The difference between `sync` and `barrier` is that `barrier` additionally enforces operation completion and visibility before entering the dissemination rounds via `quiet` or `__threadfence_system` depending on the path, and may also enforce target-side consistency afterward. As a result, both primitives have the same synchronization depth, but `barrier` provides stronger ordering and completion guarantees.

VII. MICROBENCHMARKING

In this section, we present selected microbenchmark results for NVSHMEM’s one-sided RMA and collective operations. As performance evaluation is not the primary focus of this work, we do not attempt a comprehensive study. Rather, these results are intended to highlight a few key performance characteristics of NVSHMEM.

A. Experimental Setup

All measurements in this section were collected on a CoreWeave H200 cluster. Each node contains eight NVIDIA H200 SXM5 GPUs with 144 GB HBM3e per GPU, connected

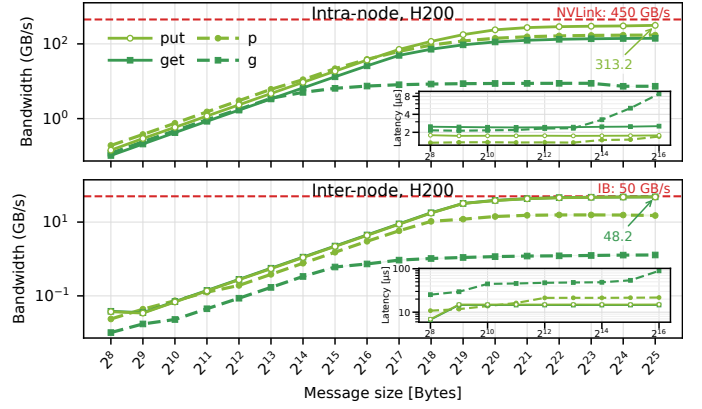


Fig. 4. NVSHMEM device-side one-sided RMA performance for bulk put/get and scalar p/g. Red dashed lines denote the hardware bandwidth upper bounds. The colored text label marks the peak bandwidth achieved in each subplot. The inter-node panel uses IBGDA-enabled runs, where the bulk put and get curves nearly overlap.

intra-node by NVLink-4. The cluster notes report 900 GB/s bidirectional NVLink bandwidth per GPU and NVLink-SHARP (NVLS) multicast support. Inter-node communication uses ConnectX-7 InfiniBand with eight NICs per node.

The microbenchmarks use CUDA 13.0.88 and NVSHMEM 3.3.9 built from NVIDIA’s public source release. For the one-sided bandwidth experiments, the data was measured with NVSHMEM device point-to-point perf tests, using 32 CTAs, 256 threads per CTA. For inter-node scalar p/g, we additionally use a tuned IBGDA configuration with `NVSHMEM_IBGDA_NUM_RC_PER_PE=64`, 64 CTAs, 1024 threads per CTA, to expose the benefit of higher GPU-initiated RDMA concurrency. The `AllReduce` results correspond to the sum operation on `float` with collective launches captured and replayed using CUDA Graphs. NCCL measurements were obtained using the official `nccl-tests` repository [28]. All results in this section are averaged over eight trials. The figures also show standard deviations, but they are too small to be visible in most cases.

B. One-sided RMA

Figure 4 reports device-initiated one-sided RMA for two intra-node P2P-reachable H200 and for one H200 GPU per node. The main axes show bandwidth, while the insets show effective transfer latency.

Intra-node bulk put reaches 313 GB/s, while get peaks at 141 GB/s. Scalar p reaches 172 GB/s because many GPU threads issue independent remote stores that can be buffered and overlapped. In contrast, scalar g stays below 9 GB/s because for each remote load, the thread needs the returned value before the operation completes, limiting the number of outstanding operations and preventing deep pipelining. These results remain below the 450 GB/s NVLink reference due to NVSHMEM address translation, synchronization, work partitioning, and protocol overheads. Across nodes with tuned IBGDA settings, bulk put and get both approach the single-rail IB reference at 48.0 GB/s and 48.2 GB/s, respectively.

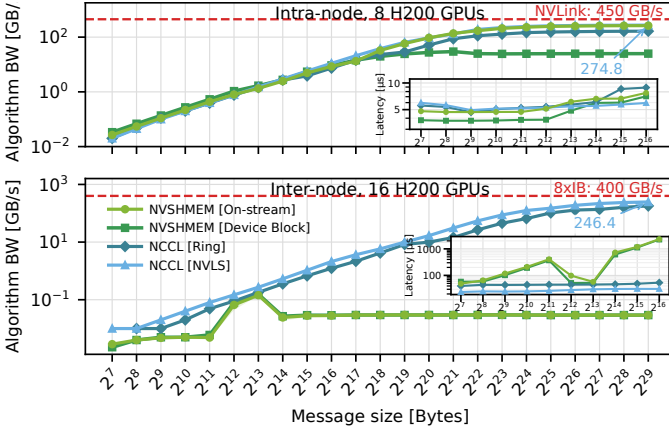


Fig. 5. AllReduce performance for NVSHMEM and NCCL. Red dashed lines denote bandwidth references: 450 GB/s for per-GPU NVLink and 400 GB/s for the inter-node case as the system has eight 50 GB/s IB NICs per node. The colored text label marks the peak bandwidth achieved in each subplot. A few inter-node NVSHMEM measurements are missing due to timeouts.

Scalar p improves substantially and peaks at 15.6 GB/s, while scalar g remains lower at 1.28 GB/s.

The latency insets show why small messages are not bandwidth efficient. Intra-node operations take about 1.8–2.5 μ s for bulk put/get and 1.3–2.2 μ s for scalar p/g . With IBGDA, inter-node put/get take about 9.4–9.5 μ s at 256 B and remain near 9.7 μ s at 64 KiB, while scalar p/g take about 7.5 μ s and 25.3 μ s at 256 B. Thus, NVSHMEM is most effective for bulk or aggregated write-style RMA, while scalar operations should be treated as latency/control primitives and batched when bandwidth matters.

C. Collective: AllReduce

For collective benchmarking, we consider only AllReduce, since it is one of the most performance-critical collectives in many applications [29]–[32] and is the operation for which both NVSHMEM and NCCL provide highly optimized implementations. Figure 5 compares their intra-node and inter-node performance. NVSHMEM [On-stream] uses the host-side `_on_stream` path, enabling multi-CTA execution and NVLS-based algorithms for the intra-node case, while NVSHMEM [Device Block] restricts execution to a fully device-initiated single-CTA path. NCCL [Ring] denotes the standard Ring AllReduce, whereas NCCL [NVLS] uses the `RSxLDMC_AGxSTMC` symmetric kernel intra-node and the NVLS Tree algorithm inter-node.

Figure 5 shows that multi-CTA execution is crucial for NVSHMEM collective performance. Intra-node, the host-side on-stream path reaches 264 GB/s in algorithm bandwidth, outperforming the forced NCCL ring baseline and approaching NCCL’s NVLS path at 276 GB/s, while the device-side block-scoped path peaks at only 30 GB/s because it uses a single CTA. For small messages, however, the device path remains latency-competitive, taking about 3.8–7.1 μ s up to 64 KiB, compared with roughly 4.7–8.9 μ s for NCCL ring and 5.6–5.9 μ s for NCCL NVLS. Across nodes, both NVSHMEM variants remain below 0.20 GB/s, while NCCL reaches 180 GB/s

with ring and 252 GB/s with NVLS Tree. This is expected as NVSHMEM’s optimized collective use has mainly focused on NVLS-based algorithms within a node or MNNVL domain. Unlike the intra-node case, the inter-node results show no small-message advantage for NVSHMEM, and its latency grows to milliseconds by 64 KiB while NCCL remains in the tens of microseconds.

VIII. CASE STUDY: DEEPEP

In this case study, we examine how NVSHMEM is used in real applications and communication libraries through DeepEP [33], an open-source library released by DeepSeek for Mixture-of-Experts (MoE) workloads under expert parallelism (EP). In EP, the experts of each MoE layer are sharded across GPUs. For every layer, tokens must be **dispatched** to the experts selected by the router, and after computation, the outputs are **combined** back to the GPUs that own the original tokens. Both phases are sparse all-to-all exchanges, where the communication volume between pairs of ranks is data-dependent. This makes them difficult to implement efficiently with off-the-shelf collective libraries. DeepEP addresses this challenge with custom dispatch and combine kernels, using NVSHMEM as the substrate for cross-node RDMA.

DeepEP exposes both a **high-throughput (HT)** path for training and a **low-latency (LL)** path for inference. In the rest of this section, we explain these two paths in more detail. Our analysis focuses on DeepEP V1, since the newer V2 backend is based on NCCL GIN and is beyond the scope of this work.

A. High-Throughput (HT) Kernels

DeepEP’s high-throughput path targets training, where large token counts make bandwidth more important than per-token latency. Its key idea is a two-stage pipeline: tokens are first sent across nodes by RDMA only between GPUs with the same local index, and are then redistributed within the destination node over NVLink to the GPUs that host the selected experts. The main components and steps of the HT dispatch algorithm are illustrated in Figure 6.

A key implementation detail is that the HT path uses *eight* parallel NVSHMEM world teams, one for each GPU slot in a node. Each world contains one PE per node, so cross-node RDMA happens only between GPUs with the same local index. However, it also assumes exactly eight P2P-accessible GPUs per node, so the design is less portable to other systems.

The communication itself is divided into two phases. The first phase, `notify_dispatch`, exchanges metadata rather than payloads. It gathers per-rank and per-expert token counts, computes prefix matrices, and determines how each channel will partition its share of the outgoing traffic. The second phase, `dispatch` (and symmetrically `combine`), moves the actual payloads. Each dispatched token is packed into a special wire format containing information such as the hidden state, optional scaling metadata, and routing information.

The dispatch kernel is organized into logical **channels**, each spanning a pair of Streaming Multiprocessors (SMs) and handling a contiguous slice of the input tokens. Within each

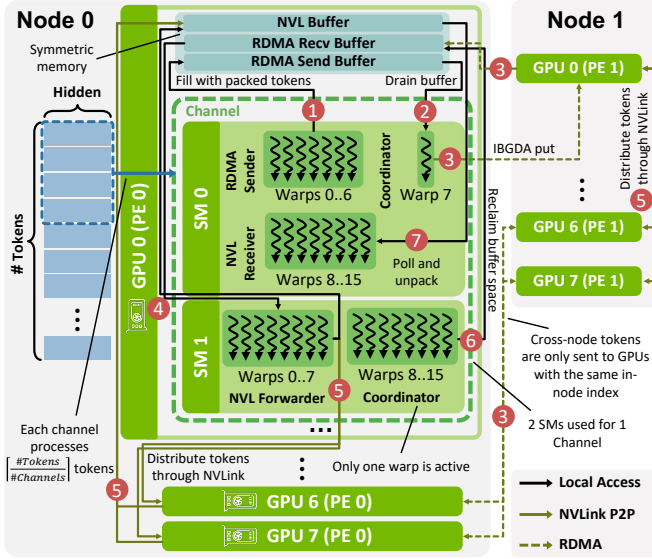


Fig. 6. Illustration of the HT dispatch algorithm for a two-node configuration. The final step in which the NVLink receiver warps store the tokens into the output is omitted. The numbered markers 1–7 show the main steps taken.

SM, DeepEP uses **warp specialization** across its 16 warps. On the RDMA-side SM, 7 warps act as senders, 1 warp serves as a sender coordinator, and the remaining 8 act as NVLink receivers, one for each local GPU slot. On the paired SM, 8 warps serve as RDMA-to-NVLink forwarders, one per destination GPU slot, while the remaining 8 are forwarder coordinators, although only one of them is active in practice.

During dispatch, the sender warps place tokens into per-peer RDMA ring buffers on the symmetric NVSHMEM heap, and the sender-coordinator warp periodically batches these writes into larger RDMA transfers through `nvshmemi_ibgda_put_nbi_warp`, updating the remote tail with `nvshmemi_ibgda_amo_nonfetch_add`. On the receive side, the 8 forwarder warps poll for arrivals, decode token metadata, and copy tokens into intra-node NVLink ring buffers. A single forwarder-coordinator warp tracks the forwarders’ progress and returns RDMA credits, while the 8 NVLink-receiver warps place the tokens into the output tensor.

The `combine` kernel follows the same structure in reverse. Expert outputs are first moved locally over NVLink into RDMA buffers, then sent across nodes, and finally reduced into the destination tensor on the token’s original GPU. The warp roles differ slightly, but the overall design is the same.

A key point is that DeepEP does not express most work as NVSHMEM collectives or generic RMA calls. Instead, it uses NVSHMEM only at critical points in the inter-node RDMA path: metadata exchange in `notify_dispatch`, chunked RDMA puts from sender coordinators, and atomic credit updates for remote head or tail counters. Thus, NVSHMEM acts as the cross-node communication substrate, while DeepEP builds its own multi-stage transport pipeline on top.

B. Low-Latency (LL) Kernels

DeepEP’s low-latency path targets inference, where batch sizes are small and end-to-end layer latency matters more than peak bandwidth. Accordingly, it removes the intra-node NVLink forwarding stage used by the high-throughput path and relies on RDMA for inter-node delivery.

Additionally, instead of partitioning the cluster into eight independent NVSHMEM world teams, the low-latency path uses a single global NVSHMEM world team and overlays a strided team for the GPUs with the same slot across nodes.

The kernel structure is likewise much simpler. There are no logical channels and warp specialization. Instead, a single kernel grid handles the whole dispatch, and SMs are partitioned by *local experts* rather than by token range. Within each block, warps are partitioned into fixed-size groups, and each warp group is responsible for one expert.

Both dispatch and combine follow this simple structure. In the send phase, each warp iterates over tokens and handles one top- k destination per warp. After optional FP8 conversion, lane 0 reserves a slot for the destination expert and computes the target address in the remote receive buffer. If the destination is on another node, the warp sends the packed message with `nvshmemi_ibgda_put_nbi_warp`; if it is on the same node and P2P-accessible, the warp instead performs a direct local copy into the mapped receive buffer. After all messages for an expert have been sent, one atomic update publishes the final token count to the destination.

In the receive phase, the corresponding warp group waits until the count becomes nonzero, reads the number of received tokens, and reserves space in the output buffer for that expert. It then copies the messages from the receive slots into the output tensor, unpacking source indices and optional FP8 scales. Overall, the critical-path NVSHMEM activity is minimal: the LL path uses an IBGDA put for payload transfer and an atomic update for the final count notification.

C. Performance

NVIDIA has already published a direct comparison of NCCL GIN and NVSHMEM on DeepEP, and since the purpose of this paper is to explain NVSHMEM rather than to add another benchmarking study, we do not repeat those experiments here. Instead, we briefly summarize the GIN paper’s results. There, DeepEP was integrated with NCCL GIN while the original NVSHMEM-based version was retained as the baseline, and both the high-throughput (HT) and low-latency (LL) dispatch and combine kernels were evaluated. The main takeaway is that NCCL GIN matches NVSHMEM closely, typically within about 1–2%, while offering the same style of device-initiated communication inside NCCL’s runtime.

IX. RELATED WORK & DISCUSSION

The closest work to ours is that of Hu et al. [5], which presents a source-level analysis of NCCL. Among prior works on NVSHMEM itself, the most directly related is that of Langer et al. [27], which explains dynamic symmetric heap allocation. While informative, that study is limited to the

memory subsystem, whereas our work analyzes the library more broadly.

A separate line of work studies NVSHMEM from the perspective of applications and runtimes. Hsu et al. [34] evaluate NVSHMEM in terms of usability and functionality, while later works use it in systems such as GROMACS and CharminG to improve scaling [35], [36]. More broadly, related work on GPU-centric programming models, such as Hamidouche et al. [37] and Unat et al. [38], examines the design space around GPU-resident communication mechanisms. In contrast, our contribution is not to propose a new runtime or survey the entire space, but to provide a detailed study of NVSHMEM.

One clear takeaway from our analysis is that limited multi-CTA support remains a weakness of NVSHMEM’s built-in collective implementation, as discussed in Section VI-D. Meanwhile, NCCL is narrowing the gap through its device API. However, the GIN paper also shows that NVSHMEM still provides a meaningful advantage as a lower-level one-sided RMA substrate [11]. NVSHMEM, therefore, remains relevant for applications that need fine-grained one-sided communication and direct device-side control.

X. CONCLUSION

NVSHMEM occupies a distinct point in the GPU communication design space by enabling GPU-initiated, one-sided communication through a PGAS model. Our source-level analysis shows how this model is implemented through symmetric memory, multiple transport paths, and a layered runtime that connects device-side operations to underlying network mechanisms. We also show that NVSHMEM’s main limitations today come from its execution model and collective implementations, which often do not fully exploit GPU parallelism. The DeepEP case study highlights NVSHMEM’s practical value in modern sparse deep learning workloads. Overall, NVSHMEM has made an important contribution by exposing a flexible, device-driven substrate for fine-grained data movement, and its design continues to drive the evolution of modern GPU communication libraries.

XI. ACKNOWLEDGMENT

The research was conducted as part of the FastTrackAI project at the Singapore-ETH Centre, which was established collaboratively between ETH Zurich and the National Research Foundation, Singapore. This research is supported by the National Research Foundation, Singapore (NRF), and the Ministry of Digital Development and Information (MDDI) under the AI Visiting Professorship (Award No. AIVP-2025-005). This work also received funding from the European Research Council (Project PSAP, No. 101002047). The authors used GPT-5.5 to assist with light editing and proofreading. All content and ideas remain the original work of the authors.

REFERENCES

- [1] T. Ben-Nun and T. Hoefer, “Demystifying parallel and distributed deep learning: An in-depth concurrency analysis,” *ACM Comput. Surv.*, vol. 52, Aug. 2019.
- [2] D. Valdez-Balderas, J. M. Domínguez, B. D. Rogers, and A. J. Crespo, “Towards accelerating smoothed particle hydrodynamics simulations for free-surface flows on multi-gpu clusters,” *Journal of Parallel and Distributed Computing*, vol. 73, no. 11, pp. 1483–1493, 2013. Novel architectures for high-performance computing.
- [3] J. Francés, B. Otero, S. Bleda, S. Gallego, C. Neipp, A. Márquez, and A. Beléndez, “Multi-gpu and multi-cpu accelerated fdtd scheme for vibroacoustic applications,” *Computer Physics Communications*, vol. 191, pp. 43–51, 2015.
- [4] M. Punke and M. Salvalaglio, “Multi-gpu fast fourier transforms in matlab (for large-scale phase-field crystal simulations),” 2026.
- [5] Z. Hu, S. Shen, T. Bonato, S. Jeaugey, C. Alexander, E. Spada, J. Dinan, J. Hammond, and T. Hoefer, “Demystifying nccl: An in-depth analysis of gpu communication protocols and algorithms,” 2026.
- [6] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, “Pytorch: An imperative style, high-performance deep learning library,” 2019.
- [7] M. Shoenybi, M. Patwary, R. Puri, P. LeGresley, J. Casper, and B. Catanzaro, “Megatron-lm: Training multi-billion parameter language models using model parallelism,” 2020.
- [8] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, “Efficient memory management for large language model serving with pagedattention,” 2023.
- [9] NVIDIA, “NVSHMEM APIs,” 2025. Documentation verified against version 3.5.19.
- [10] N. Maruyama, B. V. Essen, J. Ciesko, J. Wilke, C. Trott, C.-H. Hsu, N. Imam, J. Dinan, A. Langer, C. Newburn, and S. Potluri, “Scaling scientific computing with nvshmem,” Aug. 2020. NVIDIA Developer Blog.
- [11] K. Hamidouche, J. Bachan, P. Markthub, P.-J. Gootzen, E. Agostini, S. Jeaugey, A. Shafi, G. Theodorakis, and M. G. Venkata, “Gpu-initiated networking for nccl,” 2025.
- [12] P. Markthub, J. Dinan, S. Potluri, and S. Howell, “Improving Network Performance of HPC Systems Using NVIDIA Magnum IO NVSHMEM and GPUDirect Async,” 2022. Section: InfiniBand GPUDirect Async.
- [13] S. Potluri, A. Goswami, D. Rossetti, C. J. Newburn, M. G. Venkata, and N. Imam, “GPU-Centric Communication on NVIDIA GPU Clusters with InfiniBand: A Case Study with OpenSHMEM,” in *2017 IEEE 24th International Conference on High Performance Computing (HiPC)*, pp. 253–262, 2017.
- [14] NVIDIA Corporation, “GPUDirect RDMA Documentation,” 2026. Accessed: 2026-04-22.
- [15] NVIDIA Corporation, “DOCA GPUNetIO Documentation (v3.2.2),” 2026. Last updated: March 23, 2026; Accessed: 2026-04-22.
- [16] G. Calin, E. Derevenetc, R. Majumdar, and R. Meyer, “A Theory of Partitioned Global Address Spaces,” *Leibniz International Proceedings in Informatics*, vol. 24, 2013.
- [17] B. Chapman, T. Curtis, S. Pophale, S. Poole, J. Kuehn, C. Koelbel, and L. Smith, “Introducing openshmem: Shmem for the pgas community,” in *Proceedings of the Fourth Conference on Partitioned Global Address Space Programming Model, PGAS ’10*, (New York, NY, USA), Association for Computing Machinery, 2010.
- [18] T. Hoefer, J. Dinan, R. Thakur, B. Barrett, P. Balaji, W. Gropp, and K. Underwood, “Remote memory access programming in mpi-3,” *ACM Trans. Parallel Comput.*, vol. 2, June 2015.
- [19] J. Kraus and J. Kraus, “An introduction to cuda-aware mpi,” *NVIDIA Technical Blog*, July 2025.
- [20] Message Passing Interface Forum, “MPI: A Message-Passing Interface Standard, Version 5.0,” 2025.
- [21] CLaSS Group at Lawrence Berkeley National Laboratory, “UPC++ Wiki,”
- [22] D. Bonachea and P. H. Hargrove, “GASNet-EX: A High-Performance, Portable Communication Library for Exascale,” in *Languages and Compilers for Parallel Computing (LPC’18)*, Springer, 2018. Lawrence Berkeley National Laboratory Technical Report LBNL-2001174.
- [23] Advanced Micro Devices, Inc., “rocSHMEM - ROCm OpenSHMEM GPU library,”
- [24] A. Brooks, P. Marshall, D. Ozog, M. W. ur Rahman, L. Stewart, and R. Tom, “Intel(R) SHMEM: GPU-initiated OpenSHMEM using SYCL,” 2024.

- [25] B. Shan, M. Araya-Polo, and B. Chapman, “GICC: A high-performance runtime for gpu-initiated communication and coordination in modern HPC systems,” Apr. 2026.
- [26] C. Perry and N. Sakharnykh, “Introducing Low-Level GPU Virtual Memory Management,” 2020.
- [27] A. Langer, S. Howell, S. Potluri, J. Dinan, and J. Kraus, “Dynamic symmetric heap allocation in nvshmem,” in *OpenSHMEM and Related Technologies. OpenSHMEM in the Era of Exascale and Smart Networks* (S. Poole, O. Hernandez, M. Baker, and T. Curtis, eds.), (Cham), pp. 187–198, Springer International Publishing, 2022.
- [28] NVIDIA, “NCCL Tests,” 2026. Accessed: 2026-05-08.
- [29] J. Tang, L. Robison, M. Koop, and W. Wang, “Recent improvement to open mpi allreduce and the impact to application performance,” Sept. 2024.
- [30] M. Chrapek, M. Khalilov, and T. Hoeffler, “Hear: Homomorphically encrypted allreduce,” in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, SC ’23*, (New York, NY, USA), Association for Computing Machinery, 2023. <https://doi.org/10.1145/3581784.3607099>.
- [31] A. Weingram, Y. Li, H. Qi, D. Ng, L. Dai, and X. Lu, “xccl: A survey of industry-led collective communication libraries for deep learning,” *J. Comput. Sci. Technol.*, vol. 38, p. 166–195, Mar. 2023. <https://doi.org/10.1007/s11390-023-2894-6>.
- [32] D. Xiong, L. Chen, Y. Jiang, D. Li, S. Wang, and S. Wang, “Revisiting the time cost model of allreduce,” 2024.
- [33] C. Zhao, S. Zhou, L. Zhang, C. Deng, Z. Xu, Y. Liu, K. Yu, J. Li, and L. Zhao, “Deepep: an efficient expert-parallel communication library,” 2025.
- [34] C.-H. Hsu, N. Imam, A. Langer, S. Potluri, and C. J. Newburn, “An initial assessment of nvshmem for high performance computing,” in *2020 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*, pp. 1–10, 2020.
- [35] M. Doijade, A. Alekseenko, A. Brown, A. Gray, and S. Páll, “Redesigning gromacs halo exchange: Improving strong scaling with gpu-initiated nvshmem,” in *Proceedings of the SC ’25 Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis, SC Workshops ’25*, (New York, NY, USA), p. 1314–1329, Association for Computing Machinery, 2025.
- [36] J. Choi, D. F. Richards, and L. V. Kale, “Charming: A scalable gpu-resident runtime system,” in *Proceedings of the 30th International Symposium on High-Performance Parallel and Distributed Computing, HPDC ’21*, (New York, NY, USA), p. 261–262, Association for Computing Machinery, 2021.
- [37] K. Hamidouche and M. LeBeane, “Gpu initiated openshmem: correct and efficient intra-kernel networking for dgpus,” in *Proceedings of the 25th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming, PPOPP ’20*, (New York, NY, USA), p. 336–347, Association for Computing Machinery, 2020.
- [38] D. Unat, I. Turimbetov, M. K. T. Issa, D. Sağbılı, F. Vella, D. D. Sensi, and I. Ismayilov, “The landscape of gpu-centric communication,” 2024.